厦门大学信息学院 本科选修课
2023-2024 第二学期

模 式 识 别

Python动手课

主 讲：王程教授、陈龙彪副教授

厦门大学信息学院计算机科学与技术系

人工智能引论101计划系列课程



机器学习与模式识别

- 机器学习：使计算机系统利用数据来自动学习和改进性能的技术。
- 模式识别：识别数据中的模式和规律，用于分类和预测任务。
- 机器学习与模式识别



课程目标

- 理解机器学习和模式识别的基本概念。
- 学会使用Python进行数据处理和机器学习建模。
- 掌握几种基本的机器学习算法及其在模式识别中的应用。



为什么要动手实践

课程概览

- 数据准备：加载数据、预处理、特征工程。
- 评估指标：准确率、召回率、F1分数、交叉验证。
- 算法实践：线性算法、决策树、朴素贝叶斯、k-近邻。
- 集成方法：Bagging、随机森林、堆叠泛化。
- 实践练习：编程练习，加深理解。

Python

- Python在数据科学中的应用广泛
- 丰富的数据处理和机器学习库（如NumPy, Pandas, scikit-learn）
- 代码简洁易懂，适合快速原型开发和学习
- 社区支持和资源

1 数据准备

数据准备

- 数据准备
  - 数据质量直接影响模型性能。
  - 通过数据准备，我们可以提高算法的准确性和效率。
  - 数据准备步骤一般包括：数据清洗、转换、缩放以及数据集划分。
- 数据预处理
  - 数据清洗：移除空值和异常值。
  - 数据转换：将非数值型数据转换为数值型。
  - 数据预处理在模式识别任务中至关重要。

数据缩放

- 数据缩放
  - 数据缩放是将数据集中的数值特征调整到统一的尺度的过程。
  - 目的是为了确保所有数据在模型中具有相同的重要性。
- 为什么进行数据缩放?
  - 不同量级的数据可能导致模型训练不准确。
  - 某些算法（如支持向量机和k-近邻）对数据特征的尺度敏感。
- 数据缩放的方法
  - 归一化（Normalization）：将数据缩放到[0, 1]区间

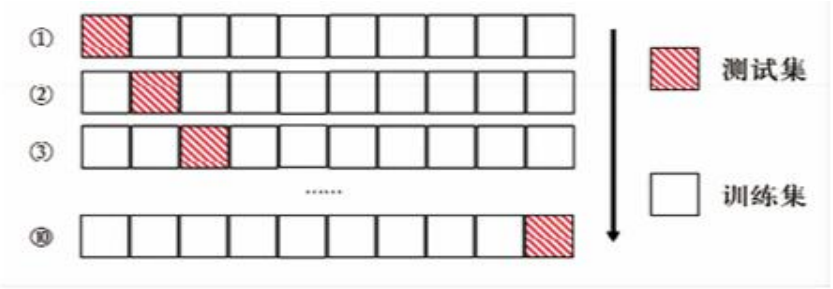
$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

  - 标准化（Standardization）：其中，μ是平均值，σ是标准差，将数据转化为均值为0，标准差为1的分布。

$$z = \frac{x - \mu}{\sigma}$$

算法评估方法

- 重采样方法
  - 重采样（Resampling）方法通过重采样可用的训练数据来评估模型。在训练时无法获取新数据，因此需要使用统计方法估计模型在新数据上的性能。
  - 这些方法包括训练集和测试集的划分以及k折交叉验证。
- 训练集和测试集划分
  - 随机划分以确保数据代表性。
- k折交叉验证
  - 将数据分成k个相等的部分来评估模型。每一部分轮流作为测试集，其余作为训练集。
  - 提供了对模型性能的更稳定和可靠的估计。



10折交叉验证原理示意图

2 评估指标

评估指标

- **Classification Accuracy**

$$\text{accuracy} = \frac{\text{correct predictions}}{\text{total predictions}}$$

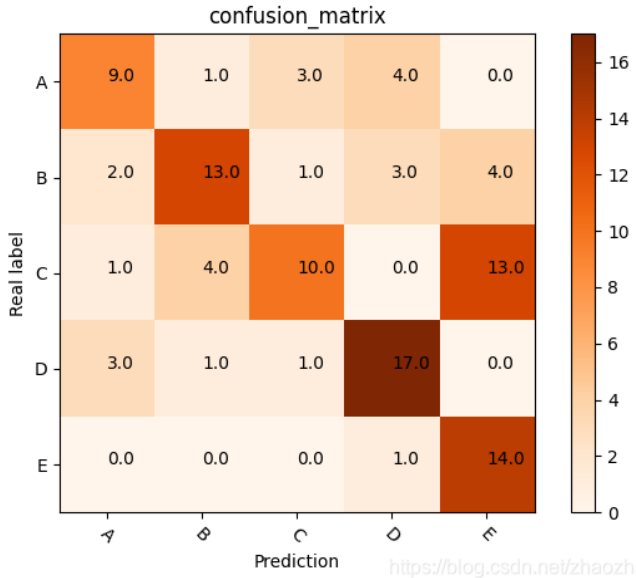
- **Mean Absolute Error**

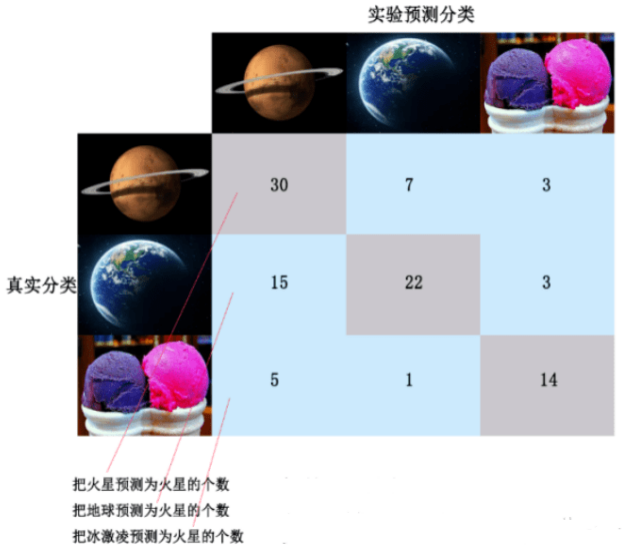
$$MAE = \frac{\sum_{i=1}^{n} abs(predicted_i - actual_i)}{\text{total predictions}}$$

- **Root Mean Squared Error**

$$RMSE = \sqrt{\frac{\sum_{i=1}^{n} (predicted_i - actual_i)^2}{\text{total predictions}}}$$

- **Confusion Matrix**





https://blog.csdn.net/zhaozhao236

评估指标

- Precision for classification
- Recall for classification
- F1 for classification
- Area Under ROC Curve or AUC for classification
- Goodness of Fit or R^2(R squared) for regression
- … …

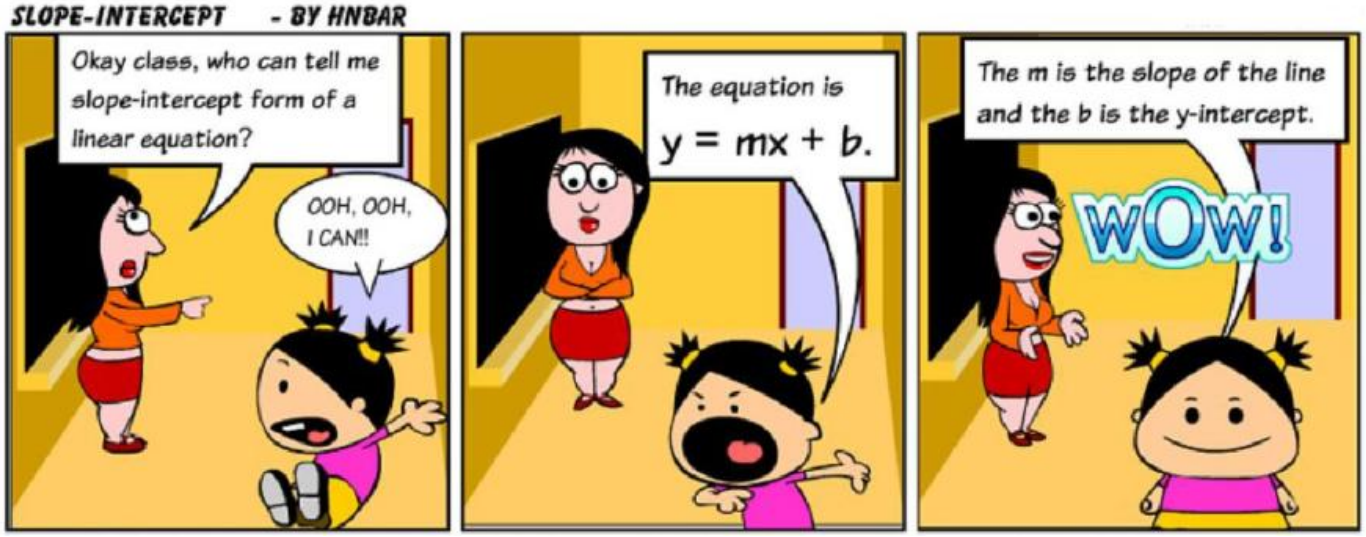
3 线性算法

简单线性回归

- 简单线性回归(Simple Linear Regression)是预测两个变量之间线性关系的方法。

$$y = w \cdot x + b$$

- 用于理解自变量如何影响因变量。
- 满足假设：线性回归假设变量之间的关系是线性的。

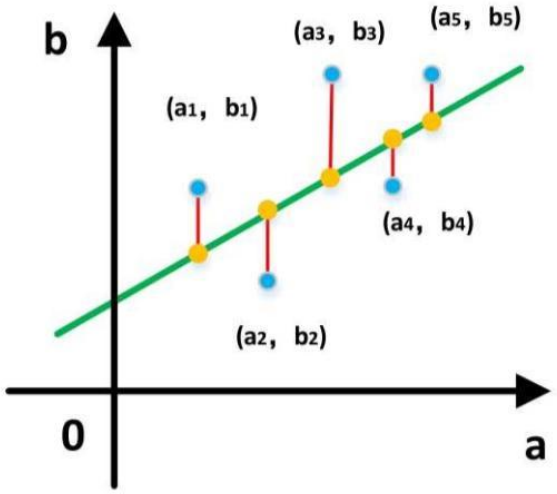


多元线性回归

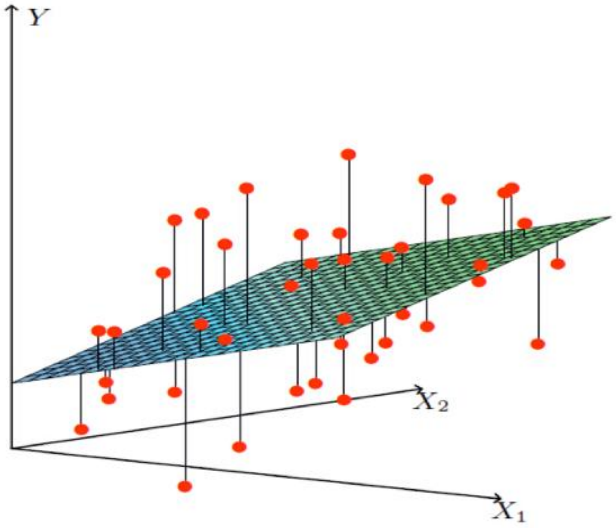
- 多元线性回归(Multivariate Linear Regression)

$$y = w_1 \cdot x_1 + w_2 \cdot x_2 + \ldots + w_n \cdot x_n + b$$

- 评估
  - 使用均方根误差（RMSE）来评估模型的预测性能



简单线性回归



多元线性回归

案例一：使用线性回归预测房价

① 数据准备

收集房屋面积、房间数、建造年份等特征，以及对应的房价作为目标变量。

② 特征工程

对收集到的特征进行清洗、转换和选择，以提取对房价有预测能力的特征。

③ 模型训练

使用线性回归算法对处理后的特征进行训练，得到房价预测模型。

④ 模型评估

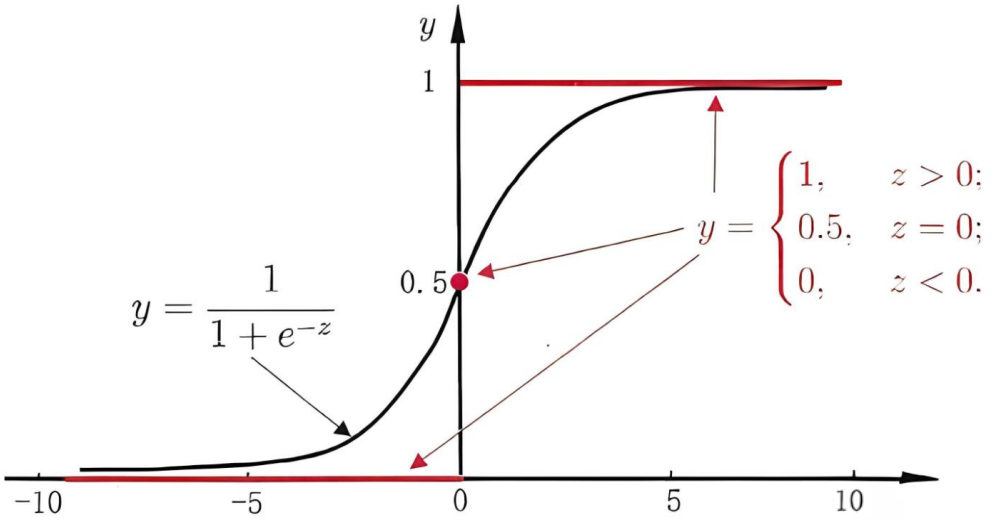
使用均方误差等指标对模型进行评估，调整模型参数以优化预测性能。

逻辑回归

- 逻辑回归(Logistic Regression)是一种分类算法，用于二分类问题。
- 用于预测一个事件的发生概率。
- 输出为概率值，通常映射到0和1之间。

$$\hat{y} = \frac{1}{1 + e^{-z}},$$

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + \ldots + w_n \cdot x_n + b$$



案例二：使用逻辑回归进行信用评分

① 数据准备

收集用户的信用历史、收入、负债等特征，以及对应的信用评分作为目标变量。

② 特征工程

对收集到的特征进行清洗、转换和选择，以提取对信用评分有预测能力的特征。

③ 模型训练

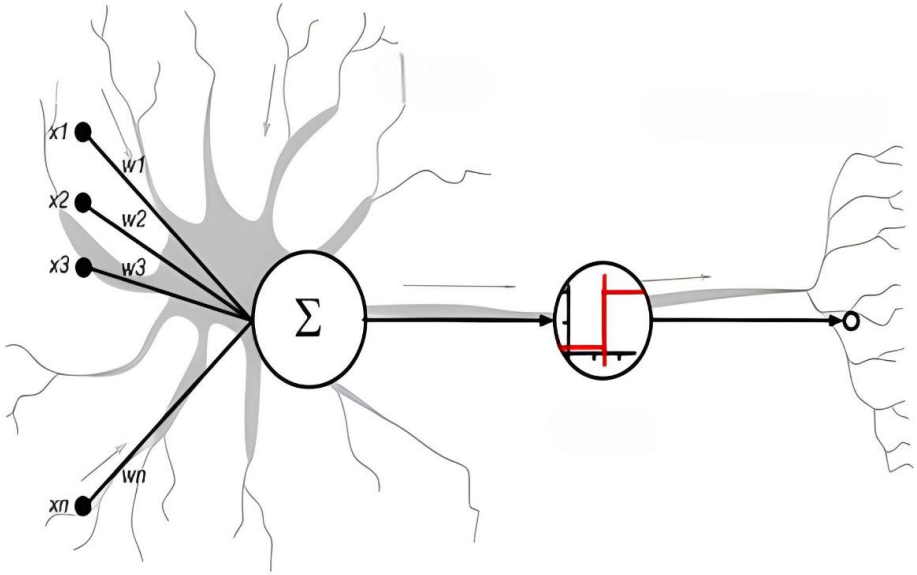
使用逻辑回归算法对处理后的特征进行训练，得到信用评分预测模型。

④ 模型评估

使用准确率、召回率等指标对模型进行评估，调整模型参数以优化预测性能。

Perceptron：最简单的神经网络

- 一个简单的数学模型，用于二分类问题，它试图模拟神经元的功能。
- 每个输入特征都有一个相关的权重，这些权重表示特征的重要性。
- Perceptron使用一个激活函数来决定输出，通常是阶跃函数，即当加权输入大于某个阈值时输出1，否则输出0。



多层感知机（MLP）



Perceptron：最简单的神经网络

学习规则

Perceptron的学习规则基于错误驱动的迭代过程。每次迭代包括以下步骤：

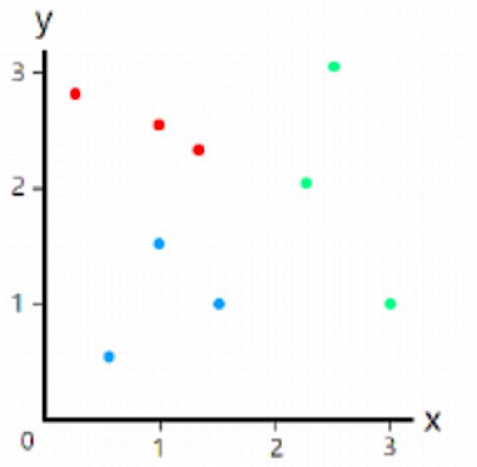
1. **前向传播**：输入特征通过它们的权重传递，并求和，形成净输入（z）。
   - $z = w_1 \cdot x_1 + w_2 \cdot x_2 + \ldots + w_n \cdot x_n + b$，其中$w_i$是输入$x_i$的权重，$b$是偏置项。
2. **激活函数**：将净输入传递给激活函数来产生输出。
   - 通常使用阶跃函数，即如果$z \geq 0$则输出$+1$，否则输出-1或0。
3. **计算误差**：将实际输出与期望输出（标签）进行比较，计算误差。
   - 误差$e = y - \hat{y}$，其中y是期望输出，$\hat{y}$是模型预测输出。
4. **更新权重**：如果产生误差，则根据误差更新权重。
   - $w_i(t+1) = w_i(t) + \eta \cdot e \cdot x_i$，其中$\eta$是学习率，$t$是迭代次数。
5. **重复过程**：重复上述步骤直到满足停止条件，例如达到最大迭代次数或误差降低到可接受的水平。

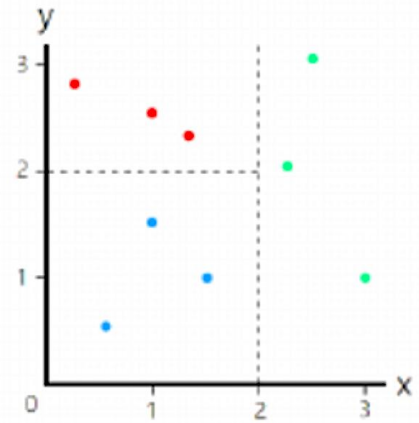
4 非线性算法

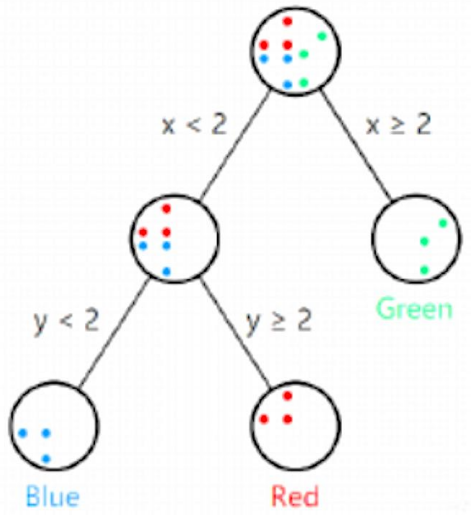
CART

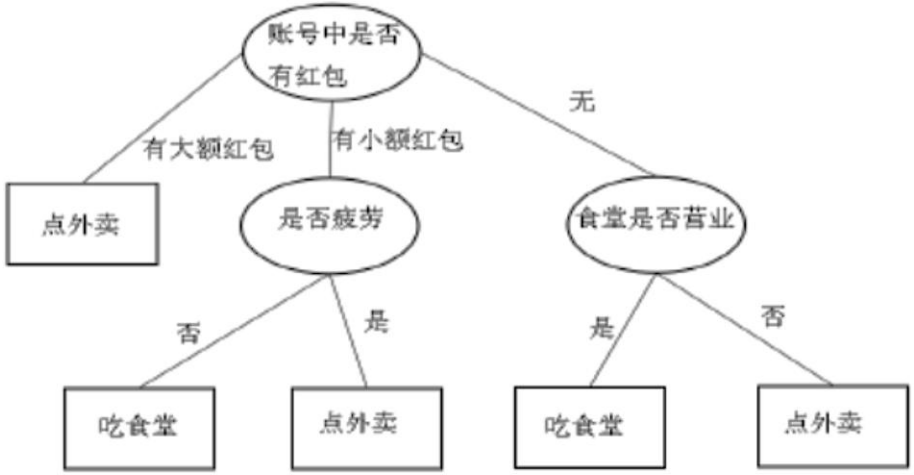
- 分类与回归树(CART, Classification and Regression Trees)可以用于分类和回归任务，决策树生成的一种算法。它模拟了人类决策过程中的决策树形结构。
- 基本结构
  - 根节点（Root Node）：包含数据集的全部数据，是树的起点。
  - 内部节点（Internal Nodes）：代表一个属性或特征，用于对数据进行分割。
  - 分支（Branches）：代表决策规则的输出，通常是“是”或“否”的问题。
  - 叶节点（Leaf Nodes）：决策树的终点，代表最终的决策结果或预测值。
- CART的构建
  - 对于分类问题，继续分割直到所有子集纯度达到某个阈值或树达到最大深度。
  - 对于回归问题，每个叶节点的输出是该节点数据的平均值。

CART


y
3
2
1
0
1
2
3
x


y
3
2
1
0
1
2
3
x


x < 2
x ≥ 2
y < 2
y ≥ 2
Green
Blue
Red


账号中是否有红包
有大额红包
有小额红包
无
点外卖
是否疲劳
食堂是否营业
否
是
是
否
吃食堂
点外卖
吃食堂
点外卖

案例三：使用CART进行客户流失预测

① 数据准备

收集客户的历史交易数据、投诉记录等特征，以及对应的流失状态作为目标变量。

② 特征工程

对收集到的特征进行清洗、转换和选择，以提取对客户流失有预测能力的特征。

③ 模型训练

使用CART算法对处理后的特征进行训练，得到客户流失预测模型。

④ 模型评估

使用准确率、召回率等指标对模型进行评估，调整模型参数以优化预测性能。

Naïve Bayes

贝叶斯定理

$$P(C_k|x) = \frac{P(x|C_k) \cdot P(C_k)}{P(x)}$$

Naïve Bayes的“Naïve”假设

- 特征之间的条件独立性，也就是说，给定目标变量的值，一个特征出现的概率不影响任何其他特征的出现概率。

$$P(x_1, x_2, \ldots, x_n \mid C_k) = P(x_1 \mid C_k) \cdot P(x_2 \mid C_k) \cdot \ldots \cdot P(x_n \mid C_k)$$

- 这个假设大大简化了计算，但在现实世界中，特征之间往往是相关的，这就是“Naïve”（天真）的由来。

Naïve Bayes

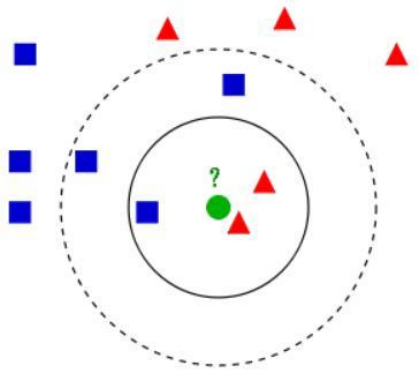
- 训练Naïve Bayes分类器
  - 计算先验概率 $P(C_k|x)$：这是每个类别在训练集中出现的频率。
  - 计算条件概率$P(x_i \mid C_k)$：对于每个类别，计算每个特征值的频率。
  - 估计证据概率$P(x)$：在实际应用中，通常可以忽略证据概率，因为它对于所有类别都是相同的，或者可以通过数据集的总数来近似。
- 预测新实例的类别
  - 给定一个新的实例 x，分类器将计算每个类别的后验概率 $P(C_k|x)$，并通过比较这些概率来选择最高概率的类别作为预测结果。
- Naïve Bayes的应用
  - Naïve Bayes算法在文本分类、垃圾邮件检测、情感分析等领域表现出色。尽管它基于一个简化的假设，但在许多情况下，它仍然能够有效地捕捉特征和类别之间的关系。

k-近邻算法

- k-近邻算法(k-Nearest Neighbors)
  - 一种基于实例的学习或懒惰学习算法。它的核心思想是：相似的样本倾向于属于同一类别。因其在预测时才进行计算，因此称为“懒惰”学习。
- 工作原理
  - **确定k值**：选择一个正整数k作为邻居的数量。k的选择对算法性能有很大影响。
  - **计算距离**：对于一个待分类的实例，计算它与训练集中所有实例之间的距离。常用的距离度量方法包括欧几里得距离、曼哈顿距离和闵可夫斯基距离。
  - **找到最近的k个邻居**：根据计算出的距离，找出距离最近的k个训练实例。
  - **进行投票**：在分类任务中，这k个邻居的类别将用于通过多数投票确定待分类实例的类别。在回归任务中，可以取这k个邻居的输出值的平均或中位数作为预测值。

K-近邻算法

- k值的选择
  - 较小的k值意味着噪声将对结果有更大的影响；
  - 较大的k值则可能导致分类边界变得模糊；
  - 实践中，常常通过交叉验证来选择一个最优的k值。
- 实现和应用
  - k-NN算法可以用任何支持距离计算的编程语言实现。在Python中，可以使用scikit-learn库中的KNeighborsClassifier和KNeighborsRegressor来进行分类和回归任务。
  - k-NN在许多领域都有应用，包括推荐系统、图像识别、生物信息学和社交网络分析等。



绿圆应该标记成红三角还是蓝方块？

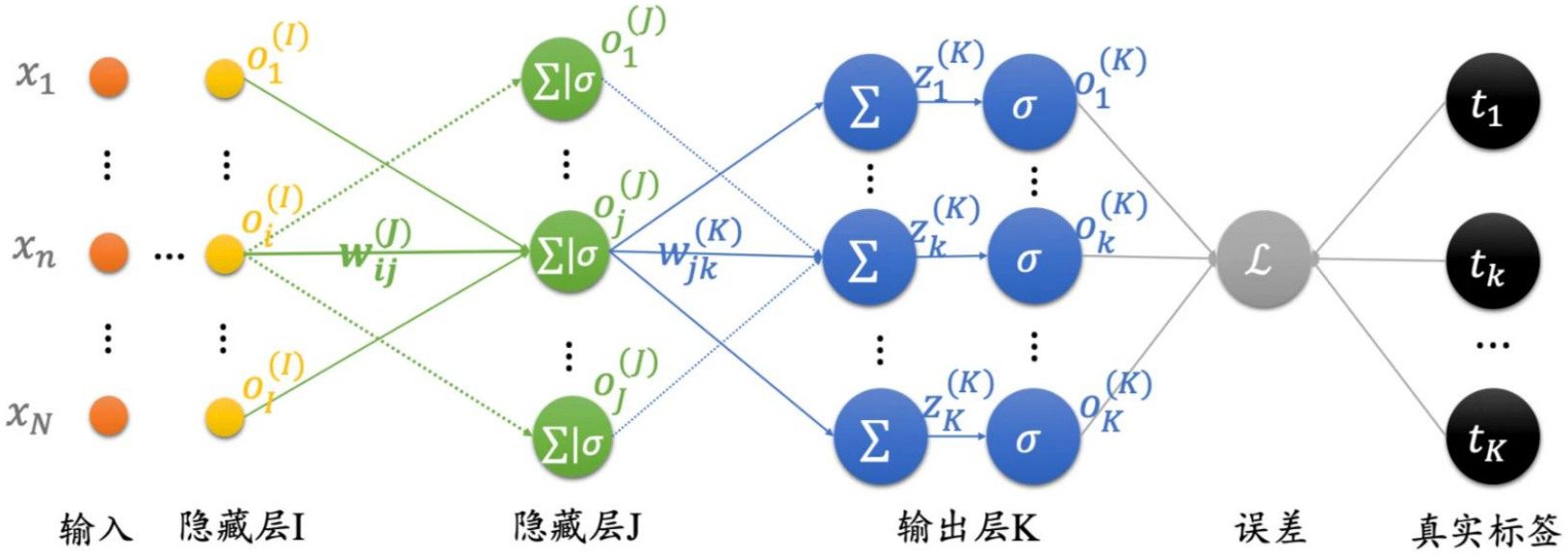
k=3 ▲

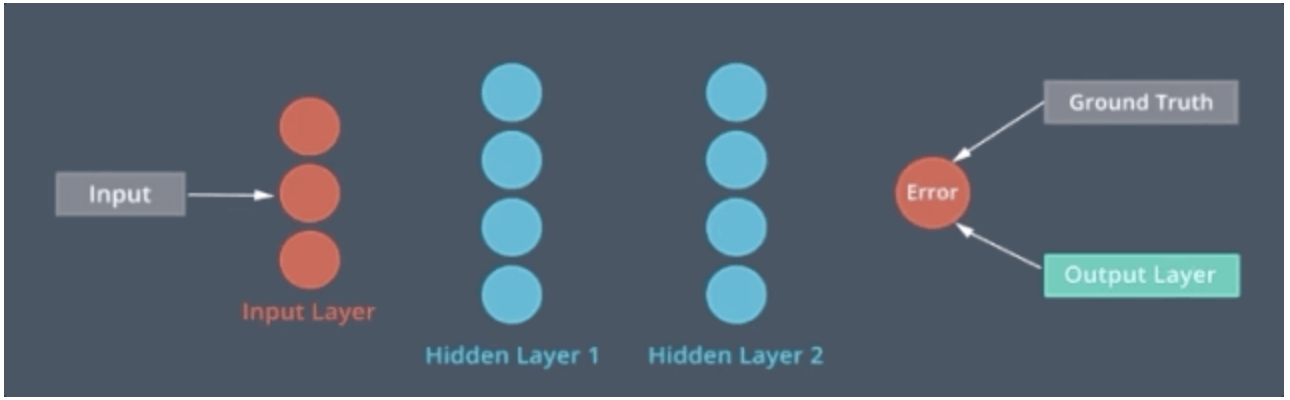
k=5 ■

反向传播算法

- 反向传播算法
  - 反向传播算法(Backpropagation)是一种用于训练人工神经网络的关键技术。它通过反向传播误差信号来调整网络中的权重，是监督学习中的一种梯度下降法。
- 基本原理
  - 前向传播（Forward Propagation）：输入信号在神经网络中从输入层经过隐藏层到输出层的传递过程。每一层的输出是下一层的输入，同时加上一个激活函数处理。
  - 损失函数（Loss Function）：衡量神经网络输出与实际目标值之间差异的函数。常见的损失函数包括均方误差（MSE）和交叉熵损失（Cross-Entropy Loss）。反向传播的目标是最小化损失函数，这通常通过调整网络中的权重和偏置来实现。

反向传播算法


x_1
x_n
x_N
$o_1^{(I)}$
$o_i^{(I)}$
$o_I^{(I)}$
$w_{ij}^{(J)}$
$\Sigma|\sigma$
$o_1^{(J)}$
$o_j^{(J)}$
$o_J^{(J)}$
$w_{jk}^{(K)}$
Σ
$z_1^{(K)}$
$z_k^{(K)}$
$z_K^{(K)}$
σ
$o_1^{(K)}$
$o_k^{(K)}$
$o_K^{(K)}$
$\mathcal{L}$
t_1
t_k
t_K
输入
隐藏层I
隐藏层J
输出层K
误差
真实标签


Input
Input Layer
Hidden Layer 1
Hidden Layer 2
Ground Truth
Error
Output Layer

反向传播算法

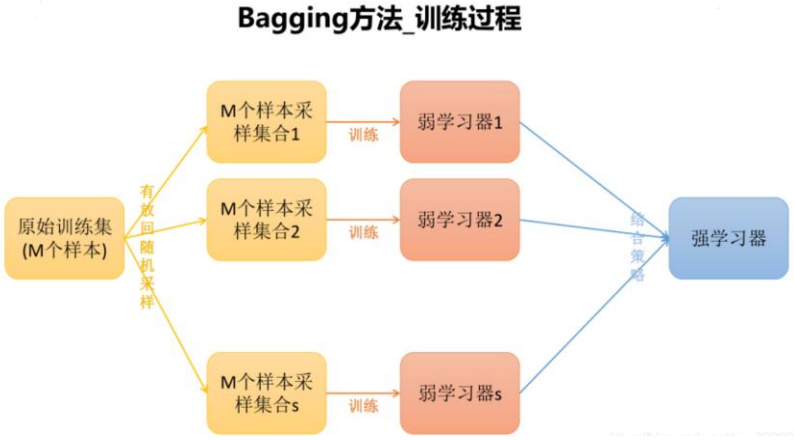
算法流程

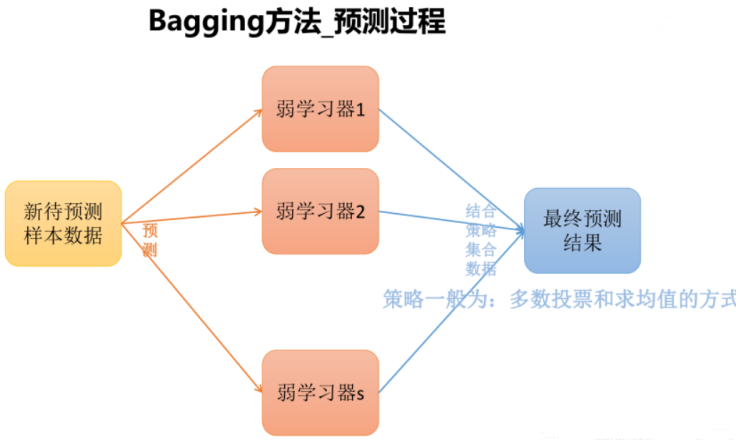
1. 初始化：随机初始化网络中的权重和偏置。
2. 前向传播：将输入数据传递到网络中，计算每一层的输出，直到得到最终的输出。
3. 计算损失：使用损失函数计算网络输出与实际目标值之间的差异。
4. 反向传播误差：从输出层开始，反向传播误差信号，计算每一层的权重和偏置的梯度。这通常通过链式法则来完成。
5. 更新权重和偏置：使用梯度下降法更新网络中的权重和偏置。更新的幅度由学习率决定。
6. 迭代：重复步骤2-5，直到达到预定的迭代次数或损失函数收敛到一个满意的值。

5 集成方法

Bagging

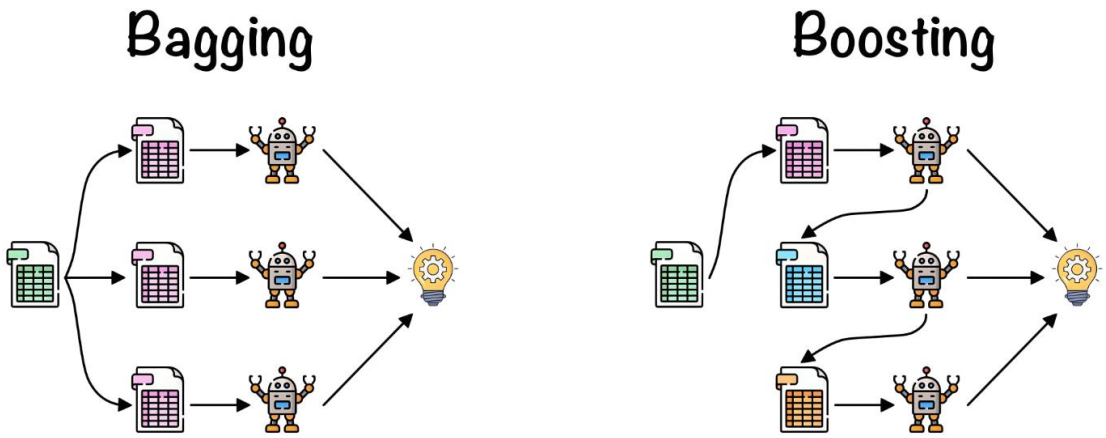
- Bagging(Bootstrap AGGregatING，袋装法)，是一种集成学习算法，主要用于提高基础监督学习算法（如决策树、神经网络）的性能。
- 工作原理
  - **样本抽取**：从原始数据集中有放回地随机选择样本，创建多个新的训练数据集，每个数据集的大小与原始数据集相同。
  - **模型训练**：对每个自助样本独立训练一个模型（基础模型），这些模型可以是相同的算法，如决策树。
  - **结果合并**：将所有基础模型的预测结果进行合并。对于分类问题，通常使用投票法（多数投票）；对于回归问题，则通常采用平均法（简单平均或加权平均）。





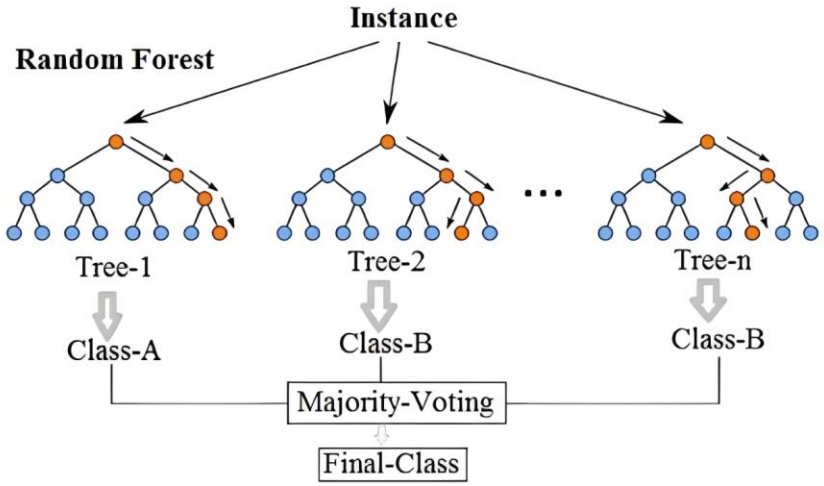
Bagging

- 优点
  - **减少方差**：Bagging可以显著降低模型的方差，尤其是对于那些高方差的模型（如未剪枝的决策树），从而提高模型的稳定性。
  - **提高准确性**：通过集成多个模型，Bagging通常能够获得比单一模型更高的预测准确性。
  - **处理过拟合**：由于每个模型都是独立训练的，Bagging有助于减少模型的过拟合风险。



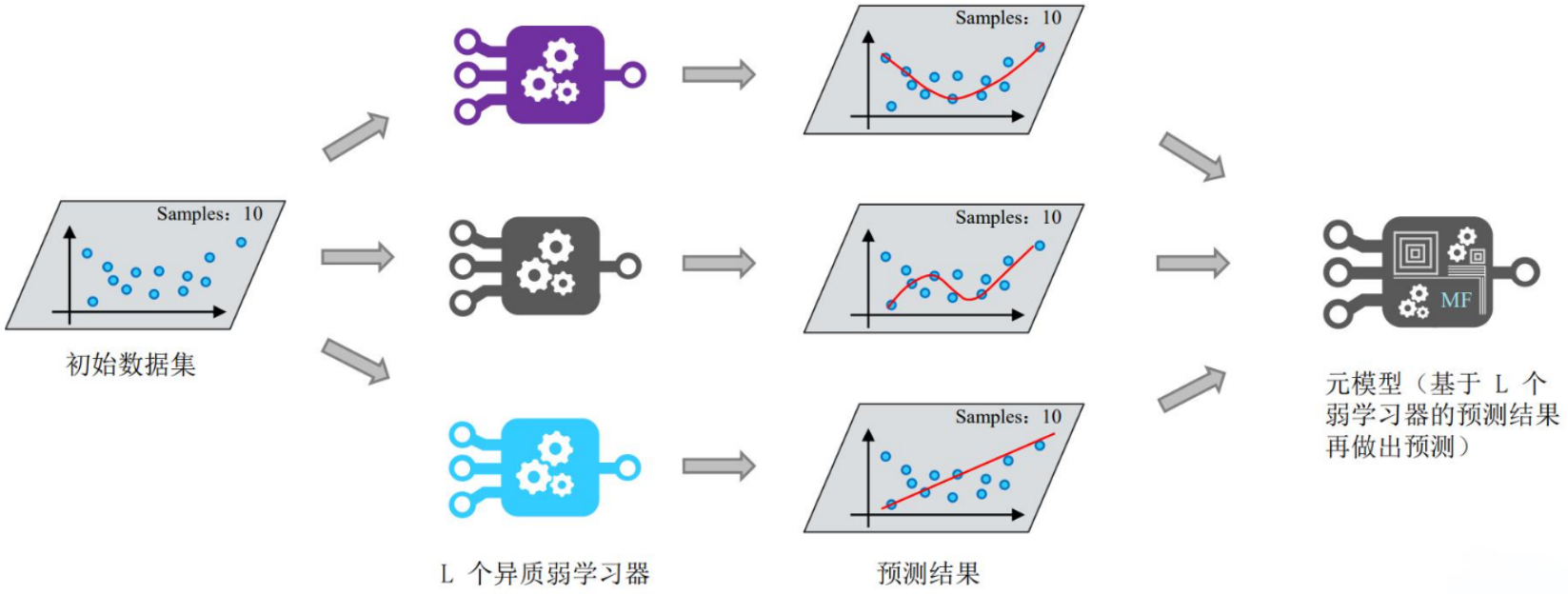
随机森林

- 随机森林（Random Forest, RF）通过构建多个决策树并对它们的预测结果进行汇总来提高模型的准确性和鲁棒性。
- 随机森林与Bagging
  - 随机森林和Bagging都是集成学习（Ensemble Learning）的方法，它们都通过组合多个弱学习器来构建一个强大的模型。
  - 随机森林是Bagging的一种特例，它使用了决策树作为基本学习器，并且在训练每个基本决策树时，对特征空间进行了随机选择，以降低模型的相关性。
  - 总的来说，随机森林是Bagging的一种扩展，通过引入特征的随机性，有效地降低了模型的相关性，提高了模型的性能。



Stacking

- 堆叠（Stacking）核心思想是使用多个基本模型（也称为基学习器）的预测结果作为特征，然后再训练一个元模型来结合这些基本模型的预测结果以产生最终的预测。
- 堆叠通过结合不同模型的优势，以期望获得更好的性能表现，从而提高预测的准确性和鲁棒性。



6 编程实践

厦门大学数字实验平台

- 网址：http://10.26.57.15
- 课程邀请码：WSYLC

End

